# Homophonic Pun Generation in Code Mixed Hindi English

**Yash Raj Sarrof**
Universität des Saarlandes
yasa00002@stud.uni-saarland.de

## Abstract

Through this project, the phenomenon of Hinglish i.e. code mixed Hindi and English, prevalent in informal online communication is explored, by focusing on pun generation. An algorithm to identify homophones from different languages and a lightweight transliteration module from Devanagari to Latin script are proposed. An existing pun generation pipeline in English and the applicability of such pipelines in code mixed settings is analysed. Finally, Large Language Models (LLMs), particularly GPT-3.5's ability to generate puns is assessed and through various prompting strategies their limitations are identified and a hybrid approach for generating puns in code-mixed settings using LLMs are suggested. Crowd-sourced evaluations validate the proposed pipeline.

## 1 Introduction

Although Hindi is the third most spoken language in the world (2), speakers who are bilingual and have English as their second language prefer to use Latin scripts in informal online settings. Conjunct consonants, vowel diacritics along with the 48 normal characters in the Devanagari script makes typing in Hindi laborious and thus, unattractive as an option. Further, bilingual speakers are familiar with the English QWERT keyboard layout and thus prefer to use the Latin script for communication in online settings. This phenomenon has led to a proliferation of the use of code mixing of the 2 languages on internet forums and in daily informal usage. Hinglish as it is colloquially called is estimated to be spoken by over 350 million people worldwide (1), and thus is a nascent hotbed for various research problems. Through this project, we try to focus on Pun generation in this code mixed setting.

Computational Humor Generation in a given language is a well studied problem, however in code mixed settings, not all methods generalise. As an example, puns based on homographs do not make any sense in code mixed settings. On the other hand, homophonic pun generation becomes all the more interesting, since different languages are bound to be using the same sound for some completely different concepts.

Large Language Models, specifically GPT line of models have been shown to be adept at generating humor and puns in English due to the presence of puns in Common Crawl and other datasets which might have been used for their training. However, recent work (3) has shown that although GPT4 can generate puns, it struggles beyond creating a small set of the same puns. This situation is further exacerbated in code mixed settings due to lack of publicly available datasets, and thus even that small set that may have been available during training, is not present, ergo leading to incoherent statements when being asked to generate them. In the project, the LLM analysed was **gpt-3.5-turbo**, and thus from hereon wherever the term LLM is used, it refers to this specific model.

The report is structured as follows, LLM's ability to generate puns is analysed in Section 2, and I try to show through Chain of Thought prompting, that the main problem it faces while generating puns is in trying to find homophones in the first place. Taking the same into account, Section 3 suggests an algorithm to identify homophones from different languages. Since Latin script is so widely used in place for Devanagari script, a lightweight transliteration module is created which achieves decent accuracy compared to state of the art despite being memory and compute efficient. The same is described in Section 4. Using both of these modules, the algorithm for pun Generation suggested in (4) is slightly modified to generate puns in the given context in

Section 5. Leveraging the best of all worlds, a pipeline to generate puns in code mixed settings using LLMs as well as our homophone identification and transliteration module is presented in Section 6. The results from a crowd sourced evaluation on our final hybrid pipeline are shown and analysed as well. Finally, the limitations and possible extensions of the work are discussed in Section 8.

## 2 CoT prompting to generate puns reveals problems with finding Homophones

Chain of Thought (5) has been shown as a simple yet effective technique in both trying to improve the outputs along with figuring out where the LLM struggles the most in carrying out a given task.

Vanilla prompting to output code mixed puns results in a code mixed sentence that is grammatical, but the results do not even attempt to make a pun and are just incoherent statements (see Figure 1). To further, test and understand the phenomenon, instructions were given on how to potentially create puns based on the He et al's(4) algorithm, and the same was tested across zero shot, one shot, and few shot settings (2, 4, 6 examples). Additionally, the prompts contained instructions for the model to output all the intermediate steps, to gauge where the difficulty lay, all the while trying to elicit Chain of Thought reasoning across all settings. The temperature of the generation was varied, and around 10 to 20 samples were generated and analysed manually in each configuration.

The general framework of the prompts designed are given in the figure 2. Thus, the model was forced to keep regenerating its output until it was able to actually find homophones. In zero shot and 1 shot learning, the model would incorrectly identify homophones and therefore the resulting pun at the end would again simply end up being a statement, instead of an actual attempt at a pun.

However, the few shot settings included examples with some negative samples of homophones, and then the rejection of those negative homophones to focus on actual homophones. A sample is shown in figure 3. Including such examples in few shot settings led to interesting cases, where the model got stuck in a recursive loop and was not able to break out of finding homophones in the first place. Eventually the maximum token limit was reached, and the api output ended with the model

```
Input: Generate a Hindi English code mixed pun

Output: Why did the Hindi teacher bring a ladder to class?
Because he wanted to teach "बंदर" (bandar) how to climb the "बांस" (bamboo)!
```

Figure 1: Vanilla Prompting leads to grammatical but incoherent statements

```
Construct a code mixed Hindi English pun based on the instructions given.
Steps
1. Create a triplet of
    a. English word "EN"
    b. Hindi word "HI" that is a homophone to the English word
    c. the translation of the Hindi word in English "HI_TO_EN".
2. If "HI_TO_EN" and "EN" are the same, redo step 1, else proceed to step 3.
3. Make short sentences (less than 10 words), with "EN" as the object of the sentence.
4. Replace "EN" with the "HI"
5. Replace the Noun Phrase at the start of the sentence with
    a contextualised phrase that is highly related to the "HI_TO_EN" word.

An example is given below
"""
Step 1:
EN: "dude"
HI: "दूध" (pronounced as "doodh")
HI_TO_EN: "milk"

Step 2:
"milk" and "dude" are not the same, proceed to Step 3

Step 3:
Short sentence with "EN" as the object: "Korbin asked - `Whats up dude ?`"

Step 4:
Replace EN with HI: "Korbin asked - `Whats up दूध (dude) ?`"

Step 5:
Output: "The cow asked - `Whats up दूध (dude) ?`"
```

Figure 2: Sample Prompt Structure explaining all steps and forcing the model to confirm the homophone before proceeding

oscillating between the 1st 2 steps for 70 turns!. A portion of the kinds of recursive outputs found can be seen in the figure 4. Therefore, it can be concluded that a major reason why these models are unable to make convincing puns is because they are unable to go past the first step of even identifying homophones. This limitation is addressed in Section 6, but before that we focus on actually identifying homophones as a separate sub task.

## 3 Homophone Identification

The Epitran package (6) available in Python can be used to convert orthographic text into their IPA forms, and its efficacy has been well studied and reported in this regard (7). The most common words found in English as well as Hindi are

```
Step 1 :
EN: "doctor"
HI: "डॉक्टर" (pronounced as "daktar")
HI_TO_EN: "doctor"

Step 2:
"doctor" and "doctor" are the same, redo Step 1

Step 1 (redo):
EN: "dude"
HI: "दूध" (pronounced as "doodh")
HI_TO_EN: "milk"

Step 2:
"milk" and "dude" are not the same, proceed to Step 3
```

Figure 3: The trace in the example section of the Prompt modified to show homophones which should be rejected

```
Step 1:
EN: "pen"
HI: "पेन" (pronounced as "pen")
HI_TO_EN: "pen"

Step 2:
"pen" and "pen" are the same, redo Step 1

Step 1 (redo):
EN: "apple"
HI: "एप्पल" (pronounced as "apple")
HI_TO_EN: "apple"

Step 2:
"apple" and "apple" are the same, redo Step 1

Step 1 (redo):
EN: "ball"
HI: "बॉल" (pronounced as "ball")
HI_TO_EN: "ball"

Step 2:
"ball" and "ball" are the same, redo Step 1
```

Figure 4: Model output stuck in a loop. This is just an excerpt, actual trace contained 70 such retries

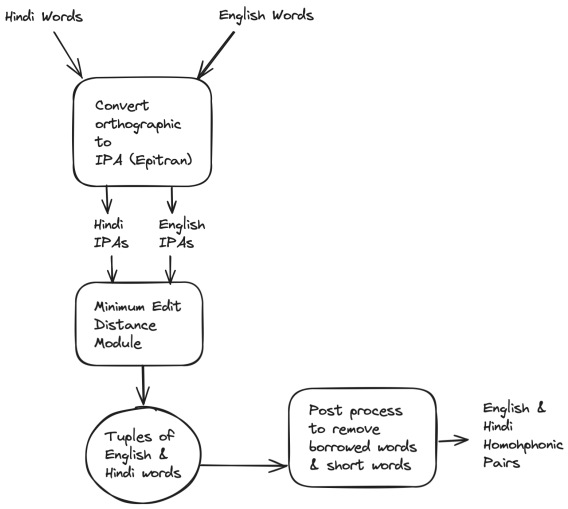


Figure 5: Homophone Identification

collected from various sources, and these words are converted into their IPA forms using Epitran. The resulting values are compared with each other and by using Minimum edit distance as a measure, similar IPAs (lower minimum edit measures) across the 2 languages are grouped with each other. This simple yet effective technique can be used to get homophones between any of the 61 languages that are supported at the moment by Epitran. However, it must be noted that some post processing is required to remove unnecessary and extra matches that might occur. Borrowed words from the same language need to be weeded out so that we are not left with simple inflections of the same word in the 2 languages. Therefore, the Hindi words are translated as well as transliterated into English. If the minimum edit distance of the translation and transliteration is low as well, then that implies that the word in Hindi was in all likelihood a borrowed word from English, and hence that word is discarded. Further, words that are smaller than 3 characters are discarded as well, since they are most likely to be filler words and would not be interesting candidates for pun generation.

## 4 Transliteration from Devanagari Script to Latin Script

As discussed in the Section 1, Devanagari script is rarely used in informal communication, hence most of the Hinglish training datasets contain transliterated text. To leverage these datasets, as well as for aiding in the post processing described in the Homophone generation step, a translitera-

| Hindi | Variation 1 | Variation 2 |
|---|---|---|
| भूख | bhook | bhuk |
| बिजली | bijli | bijlee |

Figure 6: Examples of Variations that are considered acceptable by native speakers

| Hindi | Given | Correct |
|---|---|---|
| इनके | unke | inke |
| में | be | mein |
| है | ahai | hai |

Figure 7: Some Mistakes identified in the annotation of Google's Dakshina dataset

tion algorithm was made. The rule based transliteration system achieves 64% accuracy on the Dakshina dataset (8), released by Google. The state of the art is around 72.4% (9), however that model is neural in nature and involves heavy computation. Interestingly, error analysis of the module brought into focus some general problems with such transliteration datasets. Some errors in the ground annotations were identified in the original dataset 7. A GitHub issue has been raised on the original repository to notify the authors about the mistake. Furthermore, since the script isn't native, there are no standard spellings that can be considered as the only correct one in the target script. Some examples of acceptable spelling variations are shown in the figure 6. However datasets released today generally only cover one possible variation, and thus deem the other perfectly acceptable variations as incorrect which may or may not be true.

### Transliteration Algorithm Explained

Individual character mappings for all the 42 extended set of consonants, 12 vowels, and the 12 vowel markers (attached to consonants while writing the script) to their Latin counterpart is stored locally. For any given word in devanagari script, we iterate over each of the characters. If the character is a raw vowel, and a not a vowel marker, then the latin equivalent is added as is, through our locally stored mappings. If the next character has a vowel marker, then we combine the vowel marker with the current character. The subsequent character is then checked (effective lookahead of 2). If that character turns out to be consonant with a vowel marker, then the mapping for the current character gets a $a$ along with the latin equivalent of the current character to denote the schwa sound associated with consonants *(by default every consonant of the Hindi ends with a schwa sound unless any other vowel is attached to the current vowel. However, in words, 2 schwas do not appear consecutively, hence the lookahead is applied).* If the subsequent character does not have a vowel marker, then the $a$ is not added.

**Algorithm 1:** Transliteration Algorithm for Devanagari to Latin

```
Data: devng: Word in Devanagari script
Result: latin: Word in Latin script
Initialize latin as an empty string; for
 each character c in devng do
    if c has a following vowel marker then
        c = c + vowel marker
    end
    if c is a vowel then
        Append the Latin equivalent to
         latin;
    end
    if c is a consonant then
        if there is a consonant following c
         and it also has a vowel marker
         then
            Append the Latin equivalent of
             c to latin;
        end
        else
            Append the Latin equivalent
             along with 'a' to handle the
             schwa sound to latin;
        end
    end
end
```

## 5 Modifying Pun Generation with Surprise

A stripped down version of the methodology suggested in He et al.(4) is recreated for the current context. A sample run of their execution is shown in the figure 8. Given a pair of homophones ($w_1$,

$w_2$), candidate sentences are found from a corpus (Brown corpus found in the NLTK chosen in our case). Sentences where the $w_1$ appears at the end is retained, since puns are generally considered funnier if there is an element of surprise, and the pun word appearing at the end increases the likelihood of the same. In the candidate sentences, we replace the $w_1$ with $w_2$ and subsequently try to replace the noun phrase at the start of the sentence with a topic that is related to the $w_2$ word instead of the $w_1$ word. The modified phrase results in a pun after post processing that checks for grammaticality. Since, in our case the $w_2$ is from a different language, we translate $w_2$ into English, before trying to find an appropriate topic for the same (through GLoVe Embeddings or other such distributional semantic similarity mechanisms).

**A sample execution**

- Homophone pairs: *city* , *seeti* (whistle)
- Find short candidate sentences from the Brown corpus where *city* appears at the end.
  *The man lives in the heart of the city*
- Replace *city* with the homophonic pair
  *The man lives in the heart of the seeti*
- Find apt topic to replace the subject.
  *The referee lives in the heart of the seeti*

Although the resulting phrase can lead to a pun, a filtration mechanism is required that can remove phrases / puns that don't make sense due to the change in language and can handle the grammaticality requirements of both the languages in question. This is a major limitation of this method while applying it in Code-Mixed Settings which will need to be improved upon in future extensions of this work.

## 6 Prompting LLMs by appending pregenerated list of Homophones

Since the cause of failure in GPT3.5 during pun generations was shown to be identification of homophones, the modules introduced in Sections 3 & 4 are leveraged and locally generated transliterated homophones are appended to the end of standardised prompts. A standard prompt structure is created for Zero shot, one shot, 4 shot (see Figure 9 as an example), 8 shot and finally 16 shot settings. In all cases, the input homophones are

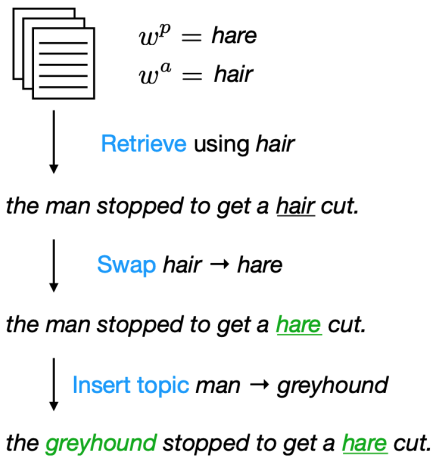


Figure 8: Sample Pun Generation mechanism. Figure taken from He et al[4]

```
Generate a Code mixed Hindi English Pun
based on the homophones provided as input.
Some example input output pairs are provided as reference.
"""
Example 1:
Input: "Submit", "Sab Mit" (everything gets erased)
Output: "Exam ki answer sheet return karte hi SUBMIT jata hai"

... more examples (if necessary)

"""
Input: "en_word", "hi_homophone_transliterated" (Translated meaning)
```

Figure 9: Standard Prompts for Hybrid approach. The sample given has been trucated to show the most interesting parts of the prompts. The last line of input is modified and replaced at run time

always appended at the end, and each input is run in a separate session without any history except for the examples in the few shot settings. This modified hybrid prompt setting improves the output drastically with many of the outputs generated fulfilling all criterion of being qualified as puns. Even, when the outputs generated are not funny, they are at least attempts at making a pun out of the given inputs.

**Survey to Evaluate Hybrid Approach**

A small survey was conducted to evaluate the quality of the puns generated by our system across all settings. A total of 56 participants volunteered for the survey and gave ratings to the puns on a scale of 1 (indicates its not funny at all) - 5

(extremely funny/ witty). Since, the survey was voluntary, the evaluation was conducted on 4-5 random samples per category. Additionally, human generated puns (collected manually from joke websites) were added to the survey to ensure nefarious or casual participation in the survey were weeded out. The puns were presented to each of the participants in a completely random order and was performed using Google Forms. It must be noted that the participants were unaware of the source of the puns to avoid bias in their judgements. The participants were between the age of 20 - 30 and were all required to be bilingual speakers of Hindi and English.

Once the survey was over, the mean ratings across each category "Human", "Zero Shot", "One Shot", "4 Shot", "8 Shot", "16 Shot" were calculated. Submissions where the mean ratings for the Human generated puns were below 3 were discarded. Additionally, submissions where the ratings for all the puns were "5" were removed as well, as most likely the participation in this case was casual and the ratings were given in jest and not full seriousness. After the filtration, a total of 39 submissions were left. The results presented in all graphs are based on calculations done based on these submissions. Some sample puns generated by the model are given in table 1

| English | Hindi | Generated Pun |
|---|---|---|
| tell | tel (Oil) | "Why did the oil go to the party? To TEL everyone how slippery it can be" |
| deal | dil (Heart) | "Why did the heart go shopping? Because it was looking for some good deals to mend its broken dil" |
| cheat | uchit (Correct) | "Why did you CHEAT in the exam?" "Mujhe UCHIT laga" |

Table 1: Some sample puns generated by the suggested hybrid prompting approach

### Mean Ratings per Category

Since humour is subjective, even the human generated puns do not always get a full score, and the mean ratings across all the samples and participants turned out to 4.32 (see Figure 10). Compared to the same, the ratings of all the other categories were pretty decent, with even Zero shot getting an average of 3.31 with slight increase in averages resulting in a 3.61 for 16 shot settings. Although these means are high, the reason for the differences in the mean with the human ratings was analysed. The top 2 puns that got the highest mean scores in each category were taken, and for each category this mean turned out to be a perfect score (see Figure 11). Conversely, while trying to plot the same thing for the puns with the lowest scores, the main difference between the human generated ones get highlighted. The mean rating for Humans drops to a 3.4 which can still be considered a pretty decent score. However, for the generated puns, the score drops down to around 1.17 to 1.5 (see Figure 12). Thus, although our model can generate a lot of good puns, there is still scope for some improvement in trying to remove the puns that become incoherent or too generic.

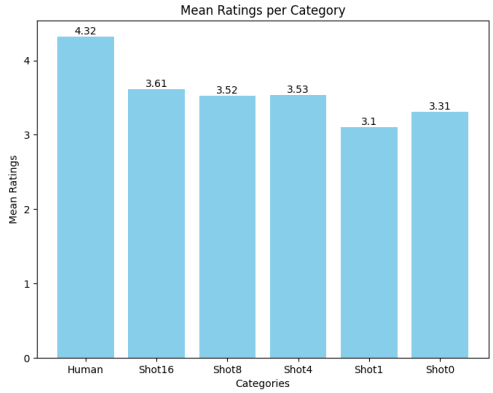


Figure 10: Mean Ratings per category of Prompts

### Comparing Against Human Benchmarks

A box plot (see Figure 13) is created to show the variability between different samples and participants in each category. When raw ratings are considered, the scores Shot 16, Shot 4 show lower variance in their scores compared to the variances shown by the one shot and zero shot settings. However if we modify our benchmark to not create perfect puns that are unanimously appreciated, but compare them against benchmarks set by human performance, since as shown from the survey, due to subjectivity of humour, it is unlikely for the mean to be a perfect 5 for a set of human generated

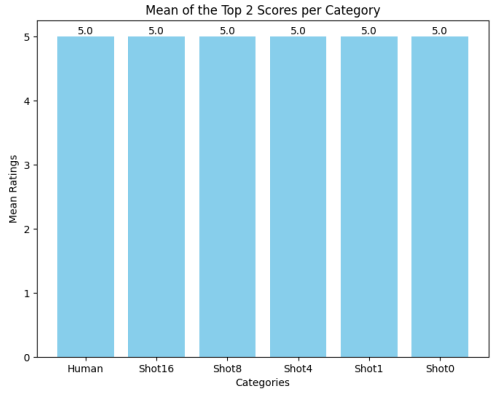


Figure 11: Mean of scores achieved by the top 2 puns per category

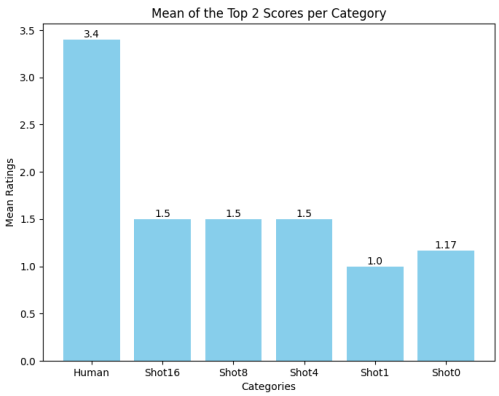


Figure 12: Mean of scores achieved by the bottom 2 puns per category

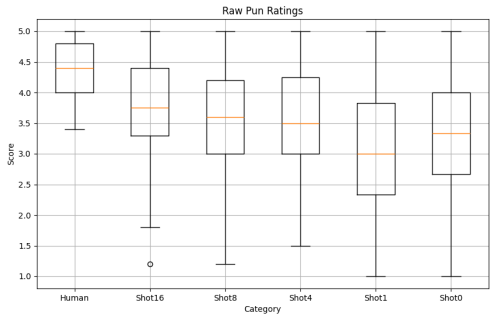


Figure 13: Box Plot with Mean Scores Per category of Prompts

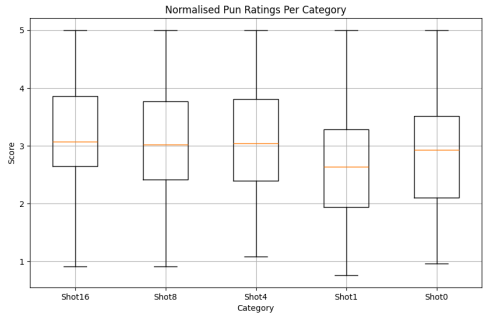


Figure 14: Box Plot with Normalised ratings according to Human Benchmarks

puns. Therefore, the mean rating given each participant to the human generated jokes was calculated, and per participant, their scores for all other categories were normalised based on this value. The resulting normalised scores are plotted as a box plot in Figure 14. It can be seen that Shot4, Shot8 and Shot16 perform the best. Zero Shot setting performs better than One shot settings, although they are both clearly inferior to the Few shot settings. All in all, the performance of such models in such hybrid settings is encouraging and further proves that LLMs can perform exceedingly well on highly specialised tasks such as generating puns in code mixed settings, provided they are leveraged appropriately and their weaknesses are well understood and addressed.

## 7 Limitations and possible Future Works

- The current homophone generation capability is constrained to single whole words. *I will see* vs *Aalsi* (lazy) are homophonic but span multiple words. Leveraging such multi word homophones would surely enhance the creativity of the generated content.

- The experimentation with prompting was conducted solely on $gpt-3.5-turbo$. However, newer and potentially more effective models are available. The choice of prompting strategies used in this study may not encompass the full spectrum of possible strategies. Future works can improve on both of these counts.

- The Pun Generation using Surprise module focuses on replacing individual nouns rather than considering the entire noun phrase, which may result in less contextually relevant replacements. Embeddings fine-tuned on Hinglish data to better handle the semantics of noun phrases, should lead to contextually appropriate replacements and thereby better quality of puns.

- The survey conducted as part of this study tested a few samples per method as the participants voluntarily took part without any monetary benefit. While the insights gained were valuable, a more comprehensive survey over larger of samples would be necessary to draw further conclusions.

Additionally, it must be mentioned that publicly available datasets of Hinglish often suffer from various shortcomings, including inadequate maintenance, annotation issues, and insufficient scale. These limitations hinder their suitability for training and evaluating AI models effectively. Addressing these challenges and improving their quality, availability and out of the box usability is essential.

## Ethical Statement

Private data of individuals were not collected during the evaluation survey and the survey was conducted completely anonymously. External datasets, packages used have been attributed to the original authors and were used only after confirming that they were distributed under licenses that permit research work.

## References


[1] GTS Translation. 2019. *Hinglish: The Biggest Language You've Never Heard Of With 350 Million Speakers*. Available at: https://blog.gts-translation.com/2019/06/12.

[2] Wikipedia List of languages by total number of speakers. https://en.wikipedia.org/wiki/List_of_languages_by_total_number_of_speakers.

[3] Sophie Jentzsch and Kristian Kersting. 2023. *ChatGPT is fun, but it is not funny! Humor is still challenging Large Language Models*. arXiv:2306.04563 [cs.AI].

[4] He, He, Nanyun Peng, and Percy Liang. 2019. *Pun Generation with Surprise*. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), edited by Jill Burstein, Christy Doran, and Thamar Solorio, pages 1734–1744, Minneapolis, Minnesota. Association for Computational Linguistics. https://aclanthology.org/N19-1172 DOI: 10.18653/v1/N19-1172.

[5] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv:2201.11903 [cs.CL].

[6] Epitran. GitHub repository. https://github.com/dmort27/epitran.

[7] David R. Mortensen, Siddharth Dalmia, and Patrick Littell. 2018. *Epitran: Precision G2P for Many Languages*. In Proceedings of the International Conference on Language Resources and Evaluation. https://api.semanticscholar.org/CorpusID:21719838.

[8] Brian Roark, Lawrence Wolf-Sonkin, Christo Kirov, Sabrina J. Mielke, Cibu Johny, Işın Demirşahin, and Keith Hall. 2020. *Processing South Asian Languages Written in the Latin Script: the Dakshina Dataset*. In Proceedings of The 12th Language Resources and Evaluation Conference (LREC), pages 2413–2423. https://www.aclweb.org/anthology/2020.lrec-1.294.

[9] Yash Madhani, Sushane Parthan, Priyanka Bedekar, Gokul Nc, Ruchi Khapra, Anoop Kunchukuttan, Pratyush Kumar, and Mitesh Khapra. 2023. *Aksharantar: Open Indic-language Transliteration datasets and models for the Next Billion Users*. In Findings of the Association for Computational Linguistics: EMNLP 2023, edited by Houda Bouamor, Juan Pino, and Kalika Bali, pages 40–57, Singapore. Association for Computational Linguistics. https://aclanthology.org/2023.findings-emnlp.4. DOI: 10.18653/v1/2023.findings-emnlp.4.